



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Conv traspuestas y autoencoders

Clase práctica 24

Motivación

- ¿Cómo podemos resolver un **ejercicio a mano** sobre **convoluciones**? En otras palabras, ¿cómo se calcula o aplica una convolución?
- ¿Y las **convoluciones traspuestas**?

Organización

Etapa	Inicio	Duración
	Sección 1	
Introducción	08:00	05'
Convoluciones traspuestas	08:05	40'
AutoEncoders	08:45	45'

Pero antes

- En convoluciones multidimensionales, se utilizan múltiples kernels para convolucionar cada capa.
- El **filtro** es la colección de kernels utilizado en la convolución de los canales del tensor de entrada.
- Cada filtro tiene $\#kernel = \#C_{in}$
- El output depende de $\#filtros$

Pero antes

Si tengo una imagen de tamaño 100 x 80x3 y lo paso por 1 filtro con un kernel 3x3 con stride 2 y padding 2. ¿Cuales son las dimensiones de salidas?

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{in} : number of input features

n_{out} : number of output features

k : convolution kernel size

p : convolution padding size

s : convolution stride size

Convoluciones traspuestas | Contexto

Nosotros ya vimos cómo **mantener** la resolución, mediante el *stride* y el *padding*.

También vimos cómo **disminuir** la resolución, mediante *pooling*.

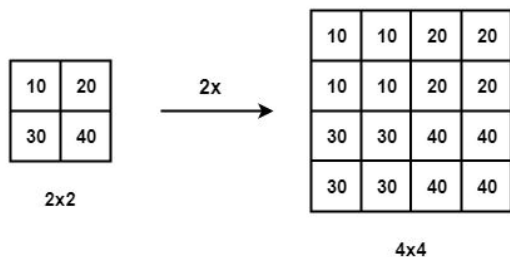
Pero, ¿cómo podemos **aumentar** la resolución?

Están las técnicas de **interpolación**, las de **extrapolación** y las **convoluciones traspuestas**.

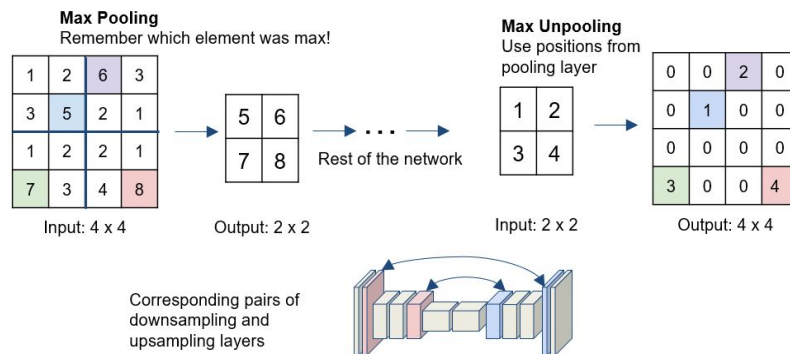
Convoluciones traspuestas | Contexto (cont.)

Recordemos que las técnicas de **interpolación** implican computar nuevos valores a partir de una **fórmula** e incluyen:

- En base a los **vecinos más cercanos**.
- **La bilineal**.



Mientras tanto, una técnica de **extrapolación** es **unpooling**. En PyTorch: `nn.MaxUnpool1d`; `nn.MaxUnpool2d` y `nn.MaxUnpool3d`; documentación oficial disponible [acá](#).



La diferencia entre interpolación y extrapolación radica en lo siguiente:

- Si el valor “rellenado” se encuentra **entre** otros dos conocidos, **interpolación**.
- Si el valor “rellenado” se encuentra a **izquierda/derecha** de los conocidos, **extrapolación**.

Convoluciones traspuestas | Concepto

Las **convoluciones traspuestas** (a veces llamadas “deconvoluciones”) tienen como objetivo **expandir el tensor de entrada**.

A diferencia de las otras técnicas, la red **aprende** un proceso de *upsampleo*, en vez de basarse en una fórmula matemática fija.

Se usan, por ejemplo, en las **GANs** (redes adversarias generativas), para aumentar las dimensiones del ruido (tamaño del bottleneck) y transformarlo en una imagen.

En el caso de **PyTorch**, la respectiva documentación oficial es [esta](#). Veámosla. Si queremos especificar sus pesos, podemos usar la [versión funcional](#) de la misma clase.

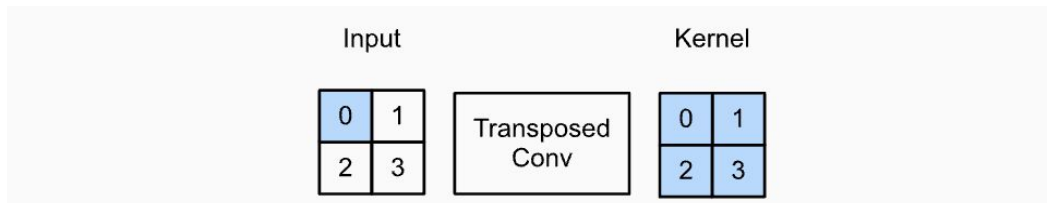
Convoluciones traspuestas | Valores

En las convoluciones traspuestas, tenemos que pensar en 3 cuestiones:

- La cantidad de **canales de salida**.
- La cantidad de **padding**.
 - El padding, en este caso, es **negativo**: se resta del *output*.
 - *Default* = 0.
- La cantidad de **stride** en el **output**.
 - El stride se aplica **al generar el output**.
 - *Default* = 1.

Convoluciones traspuestas | Un primer ejemplo

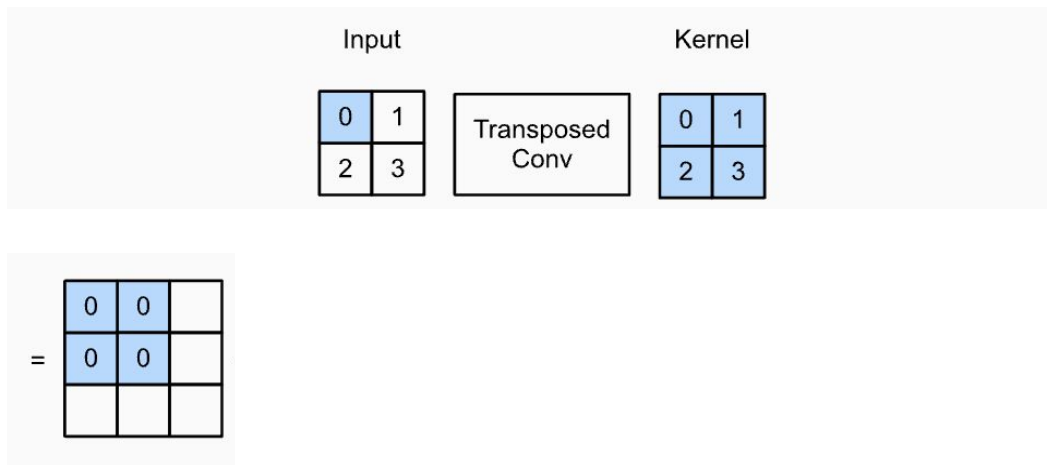
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un primer ejemplo

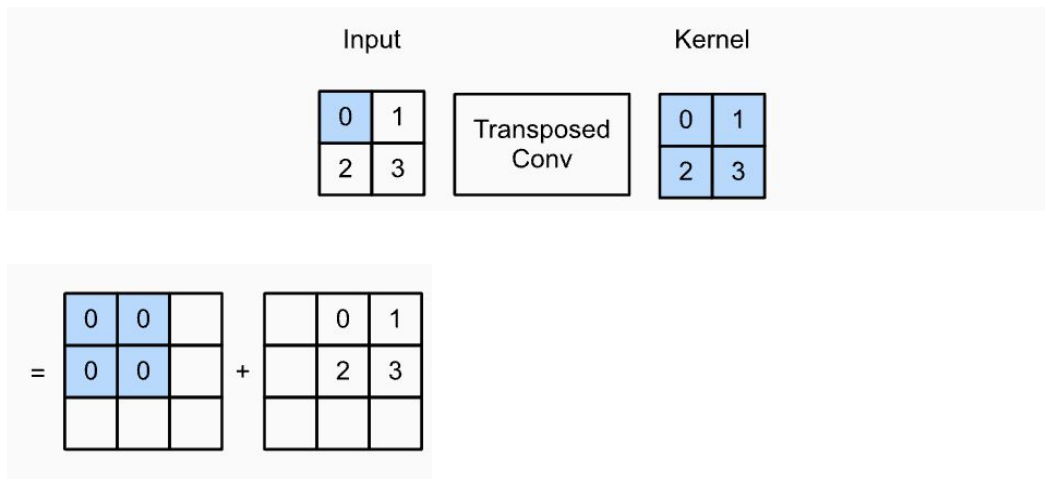
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un primer ejemplo

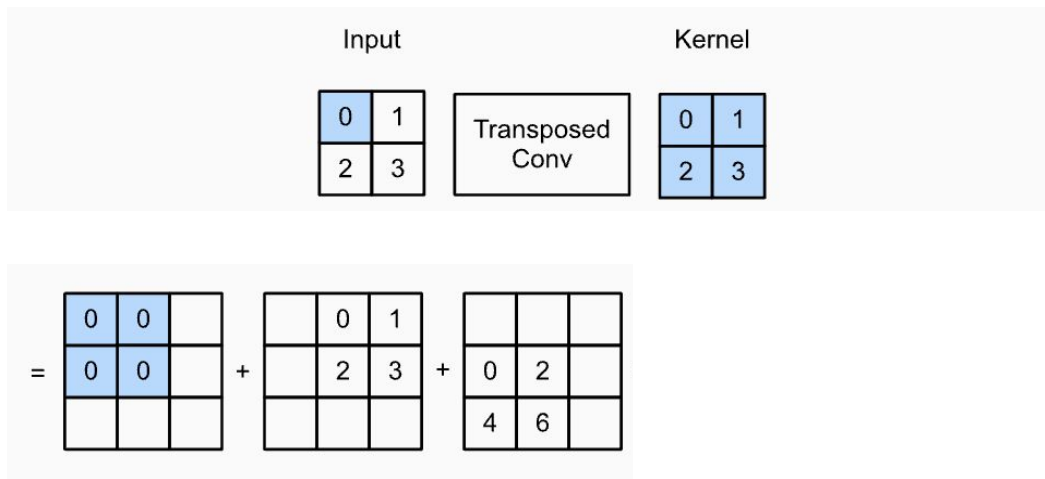
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un primer ejemplo

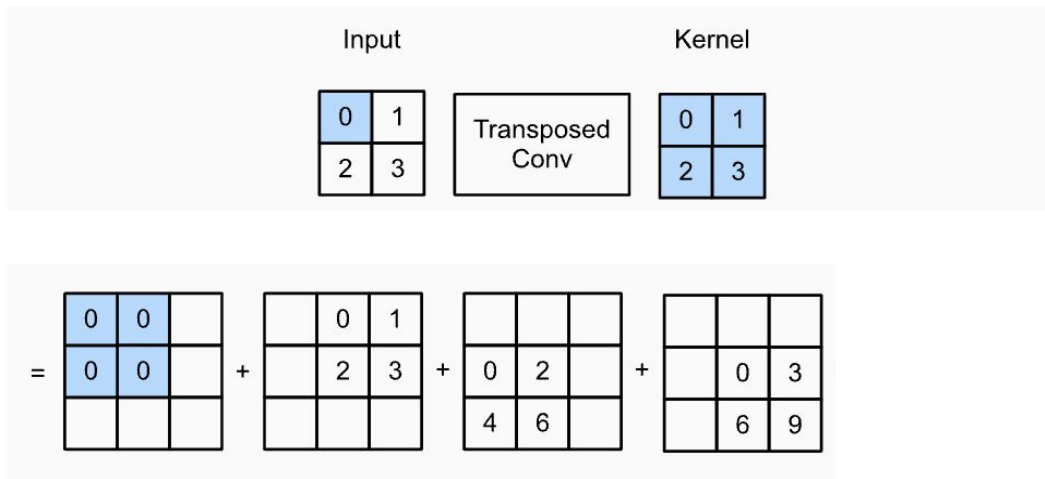
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un primer ejemplo

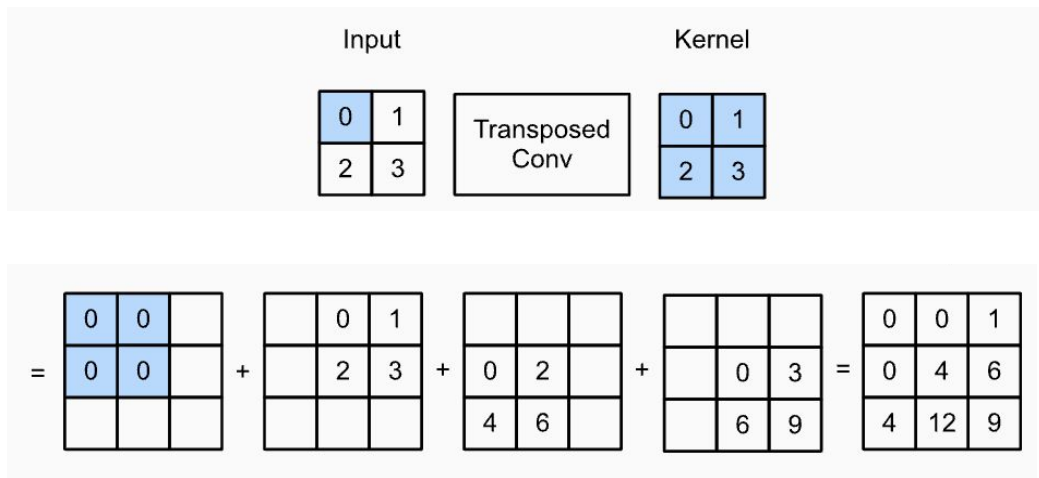
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un primer ejemplo

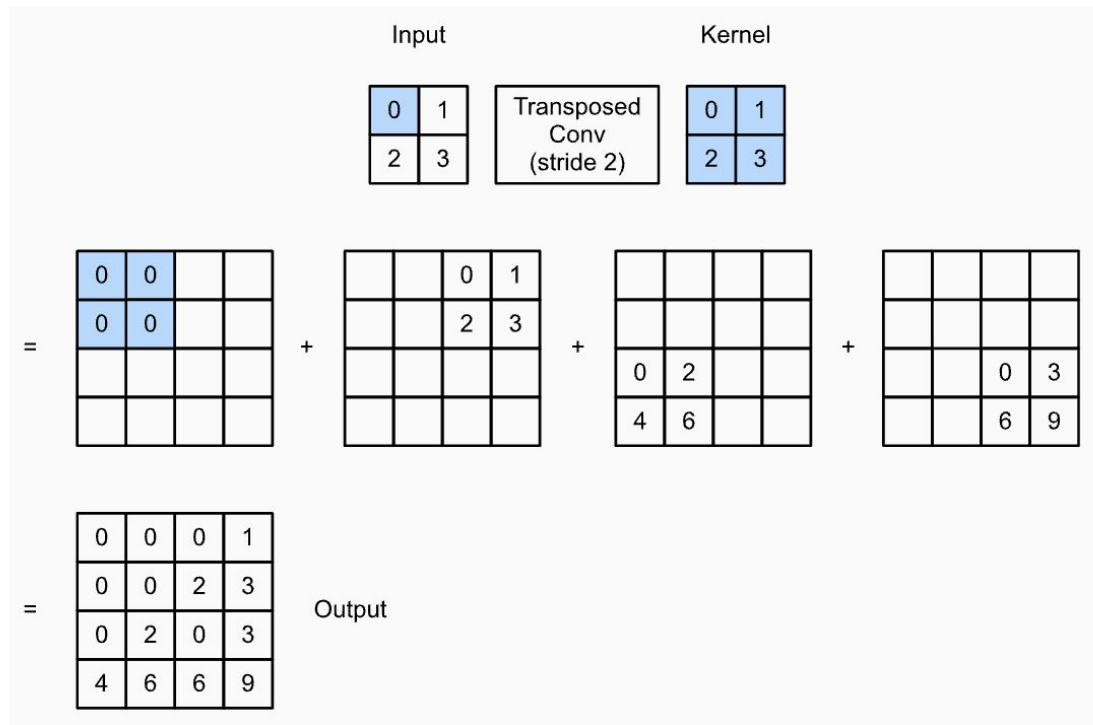
Para entender mejor la cuestión del **stride**, veamos el siguiente ejemplo, donde el padding es 0 y el stride es **1**.



[Fuente](#)

Convoluciones traspuestas | Un segundo ejemplo

Veamos otro ejemplo;
ahora con padding = 0 pero
stride = 2.



Convoluciones traspuestas | Proceso

El proceso de convolución transpuesta tiene algunos pasos adicionales:

- **Multiplicamos cada elemento del input por todo el filtro**; al hacerlo, nos vamos guardando las matrices resultantes y sus posiciones. Esta es una multiplicación escalar.
- Dependiendo del stride, **combinamos las matrices usando la suma**. Stride = 0 significaría *overlapping* total, stride = 1 es con overlapping de toda la matriz menos una fila o columna (dependiendo de la posición) y así sucesivamente.
- Finalmente, **aplicamos el padding al output**. Esto implica eliminar la cantidad de filas y columnas de los bordes con un número igual al padding. Si padding = 1, eliminamos, por un lado, la primera y la última fila y, por el otro, la primera y la última columna.

4	7	9	2
16	7	8	1
3	10	2	20
5	19	30	6

Convoluciones traspuestas | Fórmula para tamaño

Podemos calcular el **tamaño del output** con la siguiente **fórmula**:

$$\text{output size} = (\text{input size} - 1) * \text{stride} - 2 * \text{padding} + (\text{kernel size} - 1) + 1$$

[Fuente](#)

Convoluciones traspuestas | Comparación

Diferencias

Convolución

Convolución traspuesta

Convoluciones traspuestas | Comparación

Diferencias	Convolución	Convolución traspuesta
Kernel	Se multiplica elemento a elemento de cada matriz y, luego, se suman los resultados.	Se multiplica cada elemento del input <u>por todo el kernel</u> y, luego, se combinan las matrices <u>según el stride</u> .

Convoluciones traspuestas | Comparación

Diferencias	Convolución	Convolución traspuesta
Kernel	Se multiplica elemento a elemento de cada matriz y, luego, se suman los resultados.	Se multiplica cada elemento del input <u>por todo el kernel</u> y, luego, se combinan las matrices <u>según el stride</u> .
Stride	Se aplica sobre el input al hacer la convolución.	Se usa <u>en el output</u> , para combinar las matrices resultantes, después de calcularlas.

Convoluciones traspuestas | Comparación

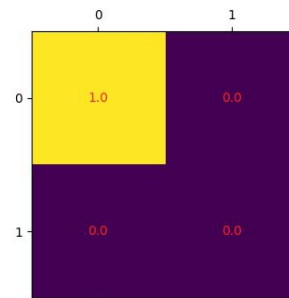
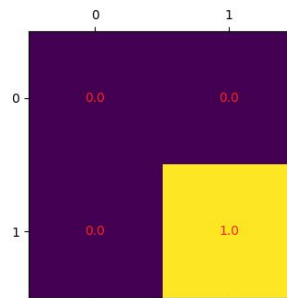
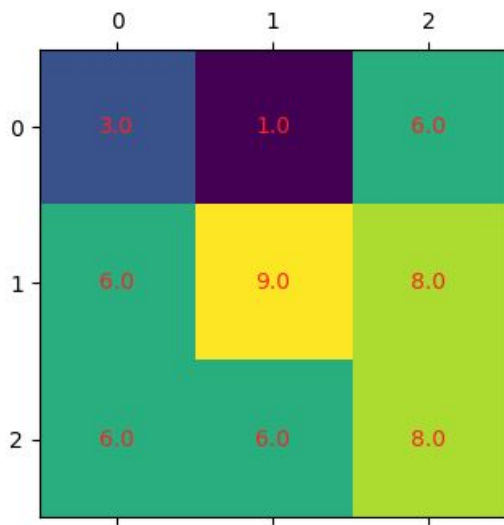
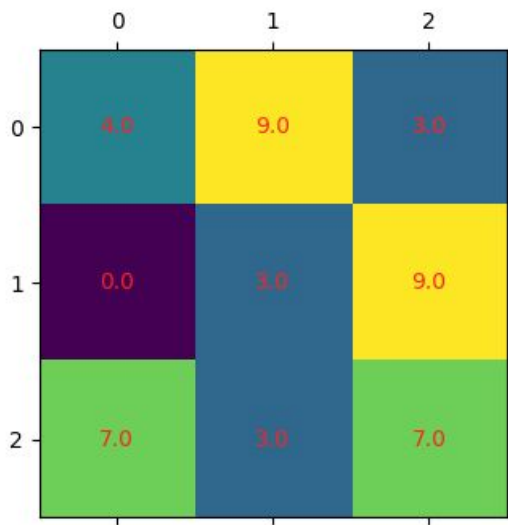
Diferencias	Convolución	Convolución traspuesta
Kernel	Se multiplica elemento a elemento de cada matriz y, luego, se suman los resultados.	Se multiplica cada elemento del input <u>por todo el kernel</u> y, luego, se combinan las matrices <u>según el stride</u> .
Stride	Se aplica sobre el input al hacer la convolución.	Se usa <u>en el output</u> , para combinar las matrices resultantes, después de calcularlas.
Padding	Se agrega al input antes de hacer la convolución.	Se aplica <u>sobre el output, eliminando</u> filas y columnas.

Convoluciones traspuestas | Comparación

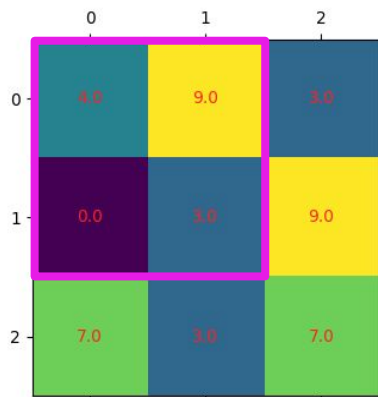
Diferencias	Convolución	Convolución traspuesta
Kernel	Se multiplica elemento a elemento de cada matriz y, luego, se suman los resultados.	Se multiplica cada elemento del input <u>por todo el kernel</u> y, luego, se combinan las matrices <u>según el stride</u> .
Stride	Se aplica sobre el input al hacer la convolución.	Se usa <u>en el output</u> , para combinar las matrices resultantes, después de calcularlas.
Padding	Se agrega al input antes de hacer la convolución.	Se aplica <u>sobre el output, eliminando</u> filas y columnas.
Tamaño de la salida	$\text{floor} [(W + 2P - F) / S] + 1$	$(W - 1) * S - 2P + F$

Convoluciones normal | Ejercicio

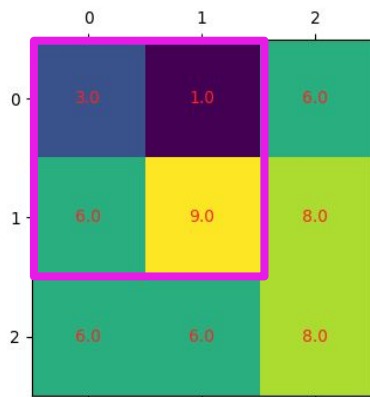
Ahora, resolvamos un **ejercicio**. Dadas las siguientes matrices de input (imagen de 3x3, con 2 canales) y el siguiente filtro convolucional normal (2 kernels de 2x2), **calcular el output de la capa convolucional**, considerando que $\text{stride} = 1$, $\text{padding} = 0$ y $\text{bias} = 0$.



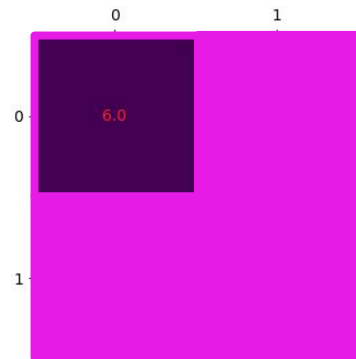
Convoluciones normal | Ejercicio



$$4 \times 0 + 9 \times 0 + 0 \times 0 + 3 \times 1 = 3$$

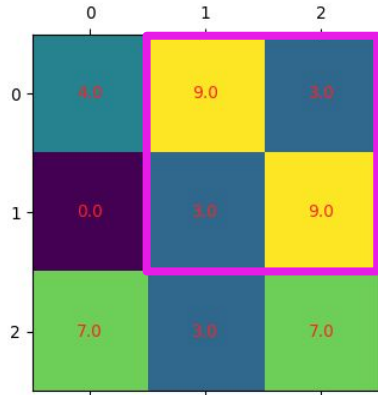


$$3 \times 1 + 1 \times 0 + 6 \times 0 + 9 \times 0 = 3$$

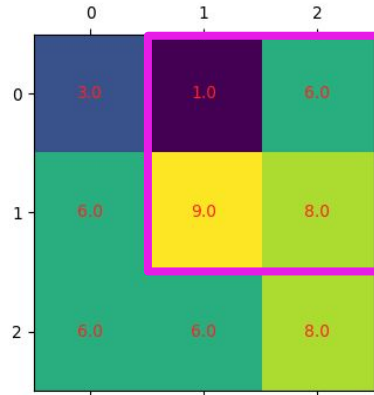


$$3 + 3 = 6$$

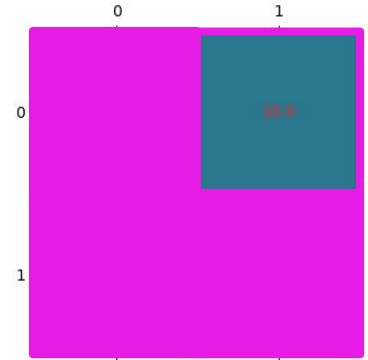
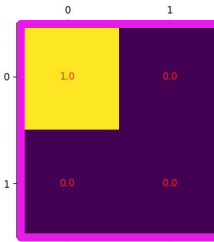
Convoluciones normal | Ejercicio



$$9 \times 0 + 3 \times 0 + 3 \times 0 + 9 \times 1 = \mathbf{9}$$

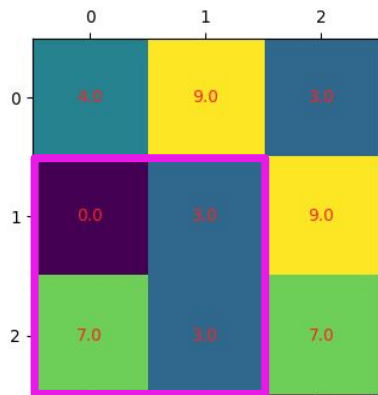


$$1 \times 1 + 6 \times 0 + 9 \times 0 + 8 \times 0 = \mathbf{1}$$

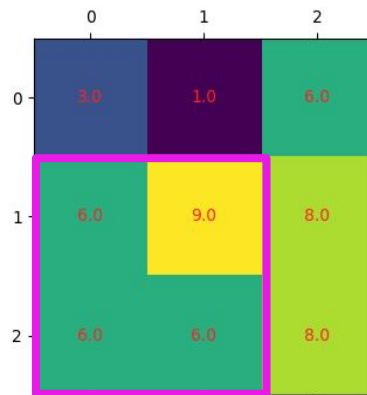
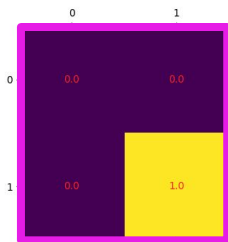


$$9 + 1 = \mathbf{10}$$

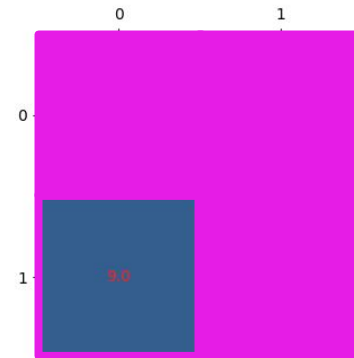
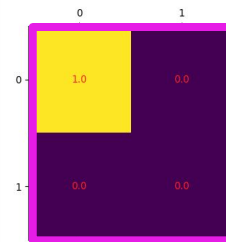
Convoluciones normal | Ejercicio



$$0 \times 0 + 3 \times 0 + 7 \times 0 + 3 \times 1 = \mathbf{3}$$

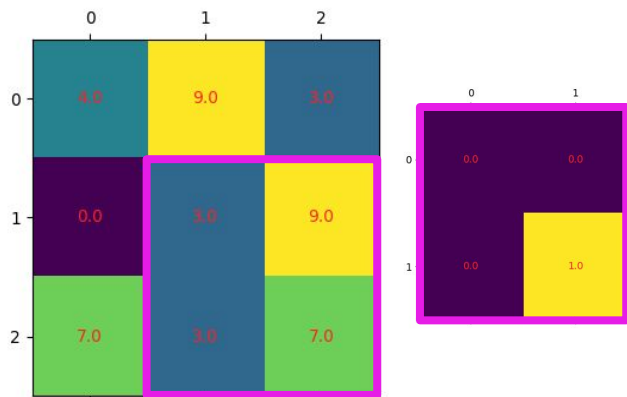


$$6 \times 1 + 9 \times 0 + 6 \times 0 + 6 \times 0 = \mathbf{6}$$

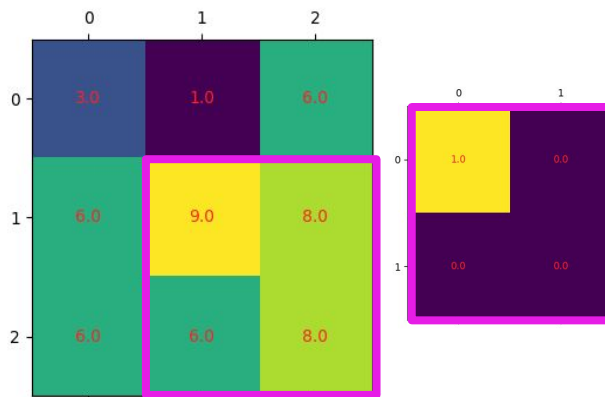


$$3 + 6 = \mathbf{9}$$

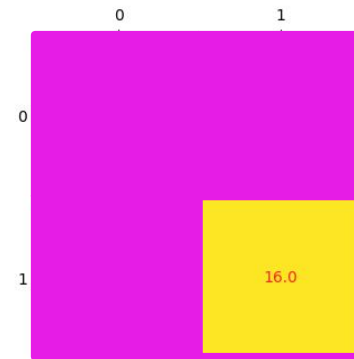
Convoluciones normal | Ejercicio



$$3 \times 0 + 9 \times 0 + 3 \times 0 + 7 \times 1 = 7$$



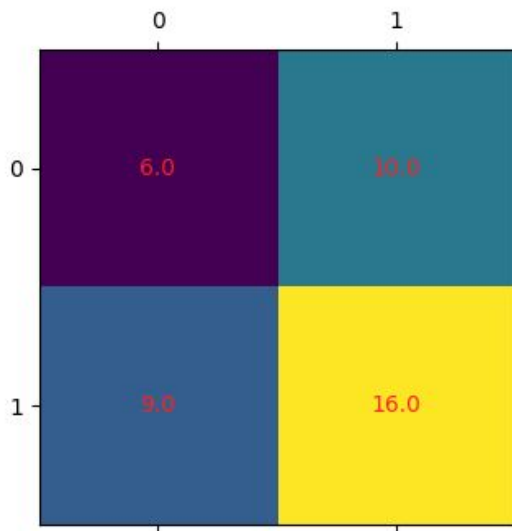
$$9 \times 1 + 8 \times 0 + 6 \times 0 + 8 \times 0 = 9$$



$$7 + 9 = 16$$

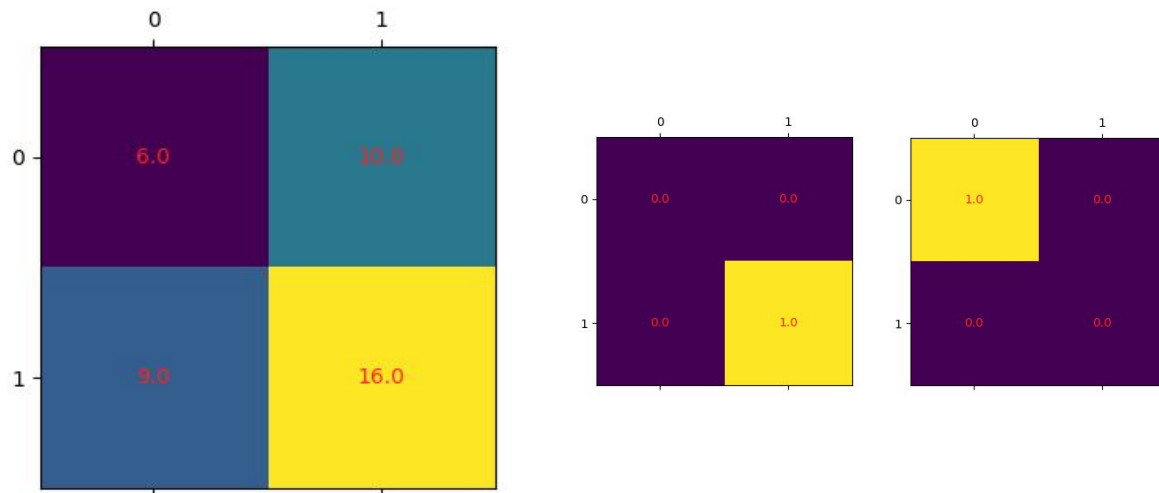
Convoluciones normal | Ejercicio

El **resultado correcto** es la imagen de abajo, de 2x2 con 1 sólo canal.



Convoluciones traspuestas | Ejercicio (cont.)

Ahora, a partir de esta imagen que obtuvimos, **calculemos la salida de dos filtros convolucional traspuesto** con los mismos kernels usados para hacer la convolución, para $\text{stride} = 2$ y $\text{padding} = 0$.



Al usar los mismos kernels, ¿esta salida será igual a la entrada original?

Convoluciones traspuestas | Ejercicio (cont.)

Primero, multiplicamos cada elemento del input por todo el primer kernel.

$$\begin{array}{cc} 6^* & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 1 \\ \hline \end{array} & 10^* & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 1 \\ \hline \end{array} & = & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 6 \\ \hline \end{array} & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 10 \\ \hline \end{array} \\ \\ 9^* & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 1 \\ \hline \end{array} & 16^* & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 1 \\ \hline \end{array} & & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 9 \\ \hline \end{array} & \begin{array}{|c|c|} \hline 0 & 0 \\ \hline 0 & 16 \\ \hline \end{array} \end{array}$$

Convoluciones traspuestas | Ejercicio (cont.)

Segundo, calculamos el tamaño de la salida, utilizando la fórmula que vimos antes.

$$\text{Tamaño de la salida} = (W - 1) * S - 2P + F$$

$$\text{Tamaño de la salida} = (2 - 1) * 2 - 2 * 0 + 2$$

$$\text{Tamaño de la salida} = 4$$

Entonces, la salida será de 4x4.

Convoluciones traspuestas | Ejercicio (cont.)

Tercero, teniendo en mente el tamaño que debe tener la salida, aplicamos stride = 2.

0	0	0	0
0	6	0	0
0	0	0	0
0	0	0	0

+

0	0	0	0
0	0	0	10
0	0	0	0
0	0	0	0

+

0	0	0	0
0	0	0	0
0	0	0	0
0	9	0	0

+

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	16

=

0	0	0	0
0	6	0	10
0	0	0	0
0	9	0	16

Convoluciones traspuestas | Ejercicio (cont.)

Cuarto, repetimos el procedimiento para el resto de los kernels.

$$\begin{array}{cc} 6^* & \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & 10^* & \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & = & \begin{array}{|c|c|} \hline 6 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & \begin{array}{|c|c|} \hline 10 & 0 \\ \hline 0 & 0 \\ \hline \end{array} \\ \\ 9^* & \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & 16^* & \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & = & \begin{array}{|c|c|} \hline 9 & 0 \\ \hline 0 & 0 \\ \hline \end{array} & \begin{array}{|c|c|} \hline 16 & 0 \\ \hline 0 & 0 \\ \hline \end{array} \end{array}$$

Tamaño de la salida: ídem (4).

Convoluciones traspuestas | Ejercicio (cont.)

Continuamos con el cuarto paso...

6	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

 $+$

0	0	10	0
0	0	0	0
0	0	0	0
0	0	0	0

 $+$

0	0	0	0
0	0	0	0
9	0	0	0
0	0	0	0

 $+$

0	0	0	0
0	0	0	0
0	0	16	0
0	0	0	0

 $=$

6	0	10	0
0	0	0	0
9	0	16	0
0	0	0	0

Convoluciones traspuestas | Ejercicio (cont.)

Entonces, la salida del filtro convolucional traspuesto en cuestión es la siguiente:

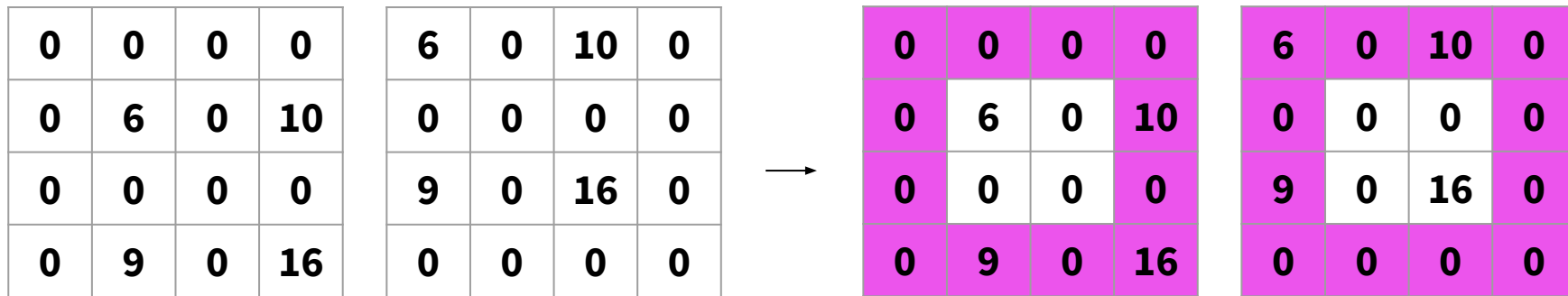
0	0	0	0
0	6	0	10
0	0	0	0
0	9	0	16

6	0	10	0
0	0	0	0
9	0	16	0
0	0	0	0

Convoluciones traspuestas | Ejercicio (cont.)

¿Y si después nos piden con **padding = 1**? ¿Cómo hacemos?

Tomamos la salida del ejercicio anterior y le aplicamos padding = 1.



6	0
0	0

0	0
0	16

Controlemos con la fórmula del tamaño de la salida.

$$\text{Tamaño de la salida} = (W - 1) * S - 2P + F = (2 - 1) * 2 - 2 * 1 + 2 = 2$$

¡Coincide!

Convoluciones traspuestas | Ejercicio (cont.)

Corroboremos los resultados mediante

`td6-p25-d-convoluciones-traspuestas.ipynb`.

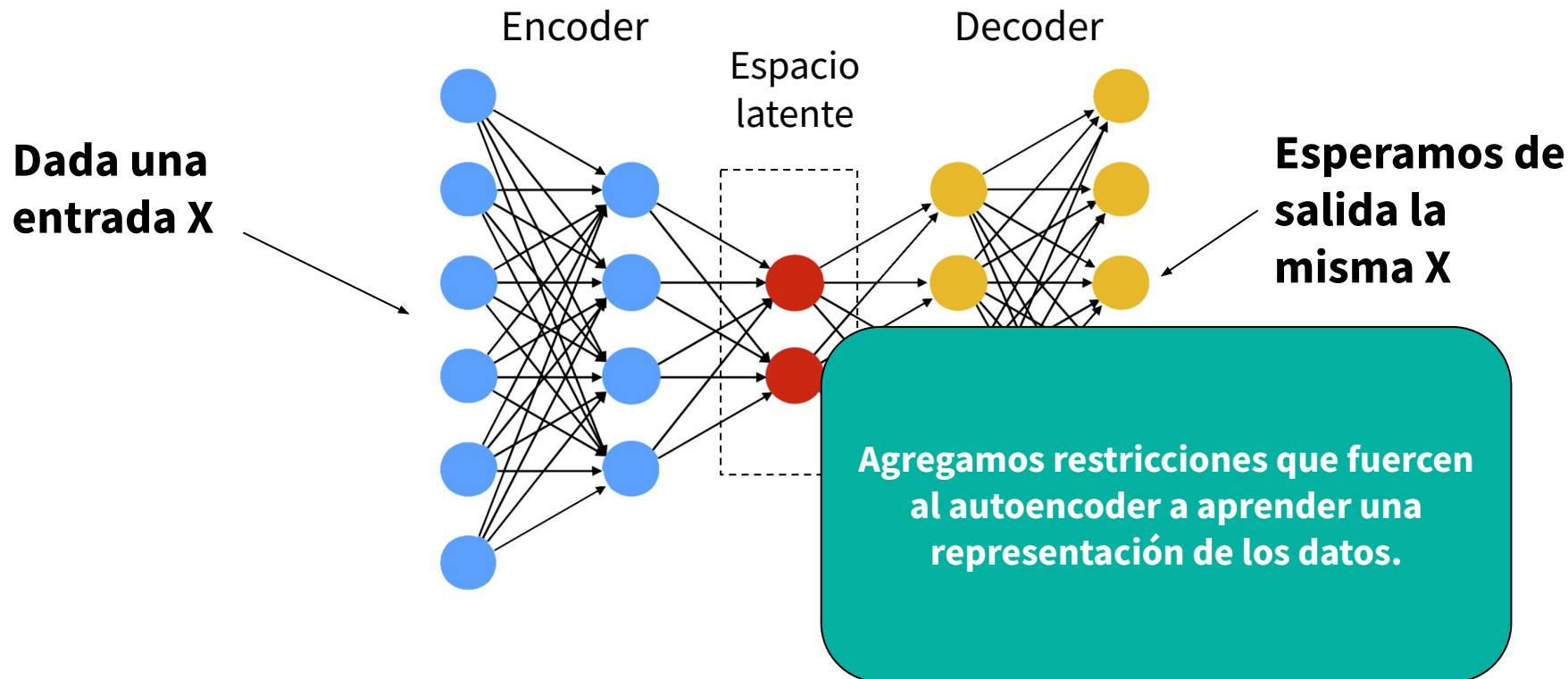
Motivación

- ¿Cómo se implementa un **autoencoder**? ¿Cómo podemos acceder a su **espacio latente**?
- ¿De qué manera podemos **eliminar ruido** en imágenes?
- ¿Cómo podemos implementar **segmentación de imágenes**?

Autoencoders

¿Qué pasa si la salida final de un MLP es una entrada para otro MLP invertido?

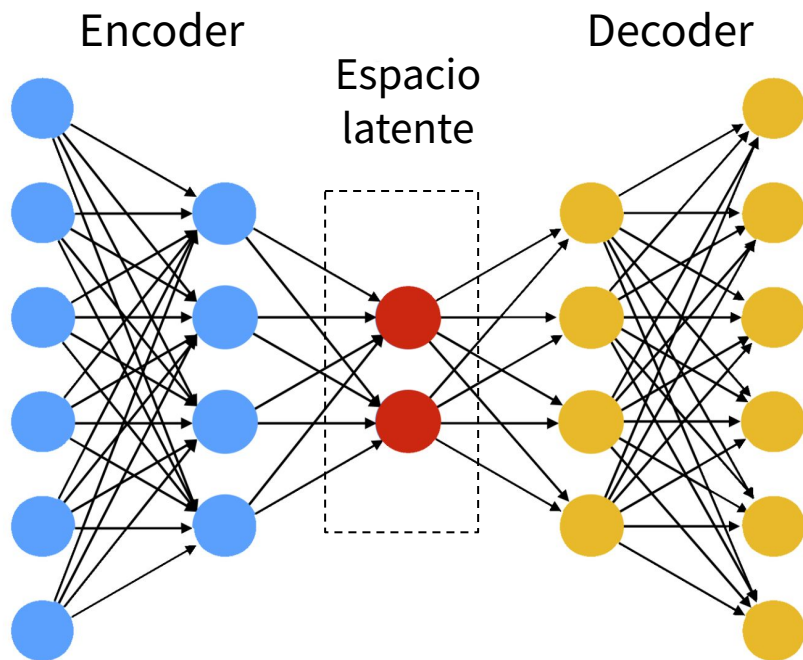
Autoencoders (continuación)



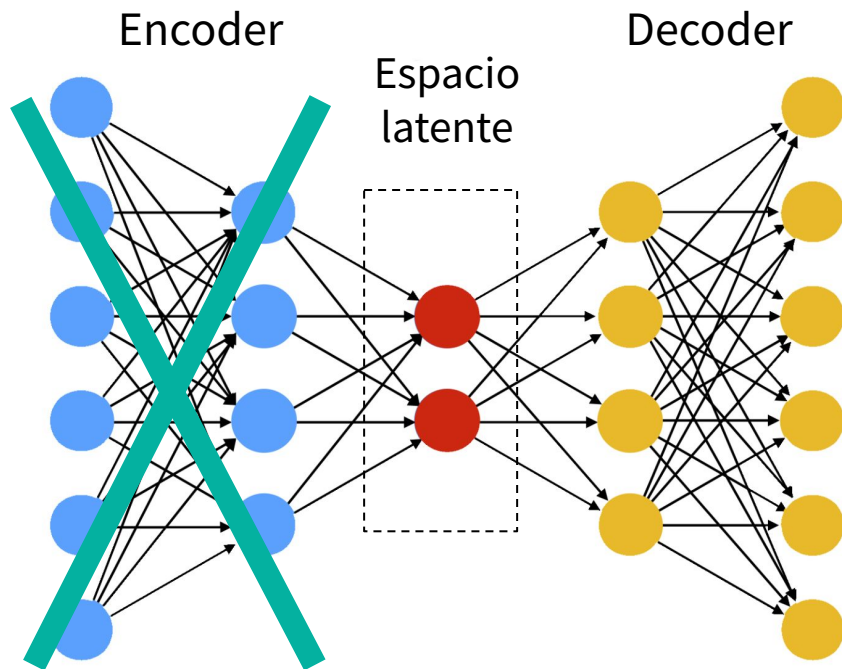
Autoencoders (continuación)

Veamos un ejemplo en `td6-p24-c-autoencoder.ipynb`.

Autoencoders (continuación)

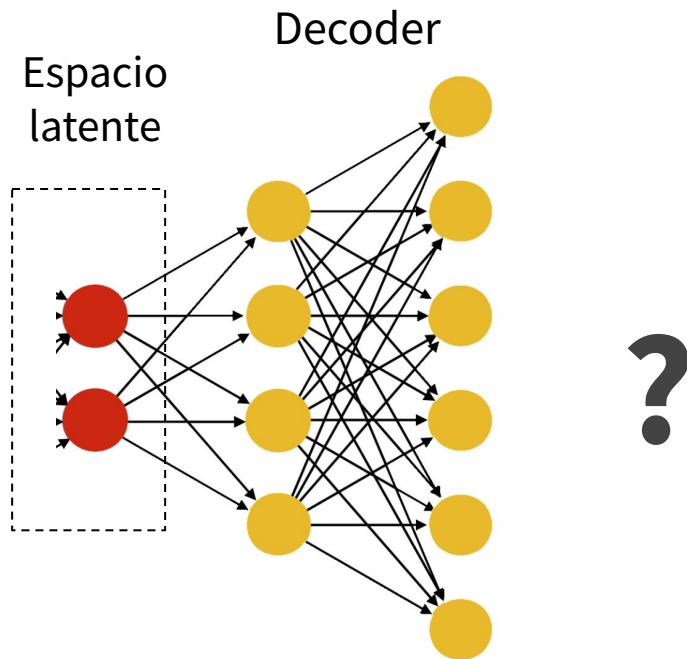


Autoencoders (continuación)



Autoencoders (continuación)

Si en este espacio latente está la representación de las imágenes, podríamos probar manipular estos valores para construir nuevas figuras.



Volvamos a `td6-p24-c-autoencoder.ipynb`.

Denoising Autoencoders (DAEs)

Problema:

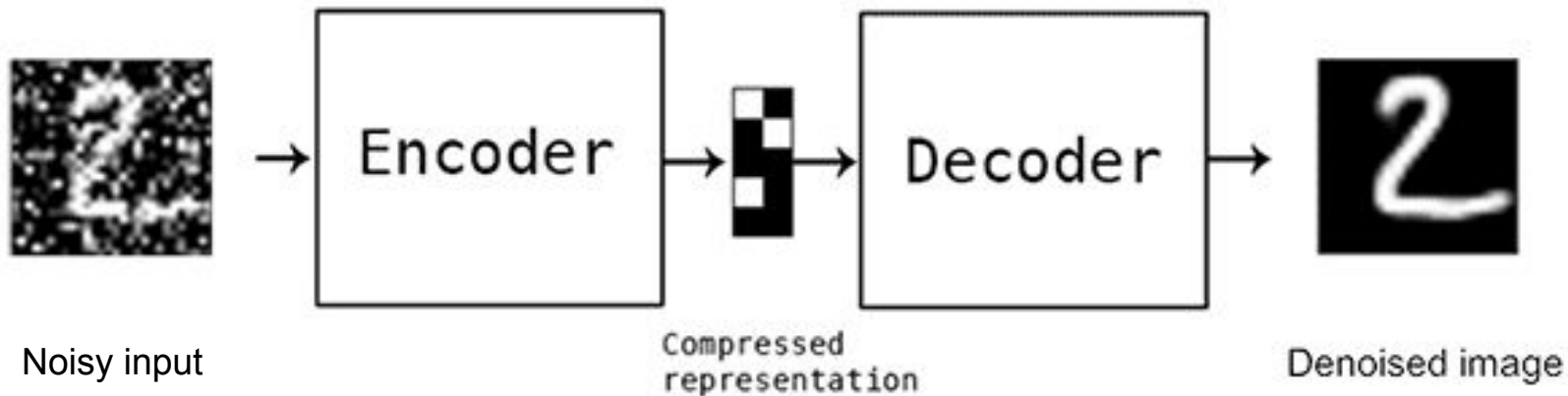
Los datos pueden tener perturbaciones no deseadas. Por ejemplo, fallas en sensores de una cámara, mala iluminación, interferencia, etc.

Denoising Autoencoders (DAEs) (continuación)

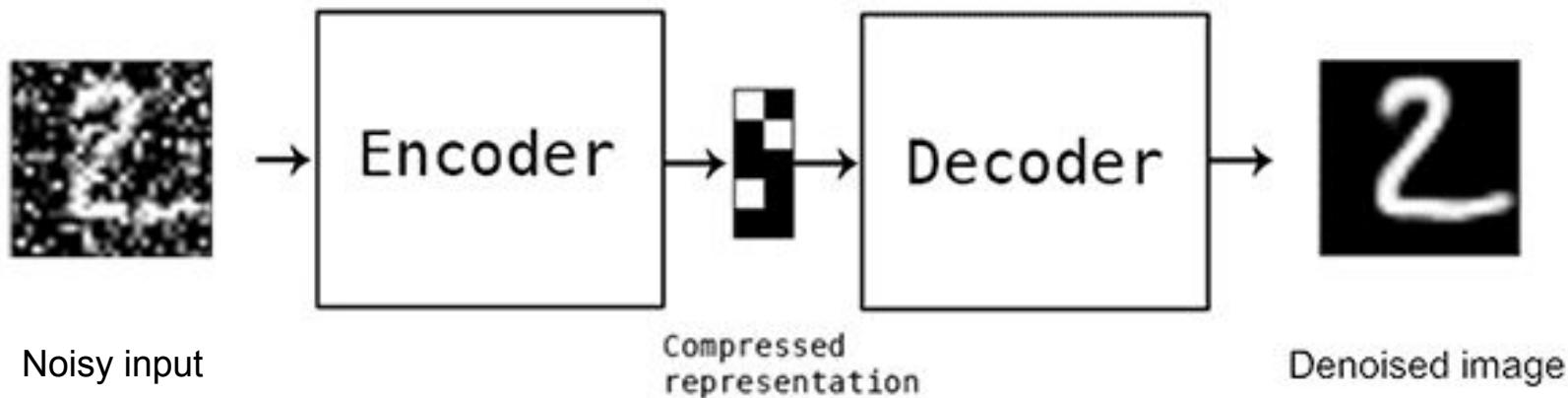
Llamamos **ruido** a las perturbaciones sobre los datos que contaminan el contenido de la información que se quiere recuperar.

Idea: Usar la estructura del Encoder/Decoder para aprender las características más importantes y luego reconstruir la imagen original.

Denoising Autoencoders (DAEs) (continuación)



Denoising Autoencoders (DAEs) (continuación)



En vez de entrenar con los elementos del *dataset* directamente, se modela el ruido. Es decir, se lo caracteriza numéricamente. La salida esperada de la red corresponde al X sin ruido.

Denoising Autoencoders (DAEs) (continuación)

Algunas **técnicas** para **agregar ruido**:

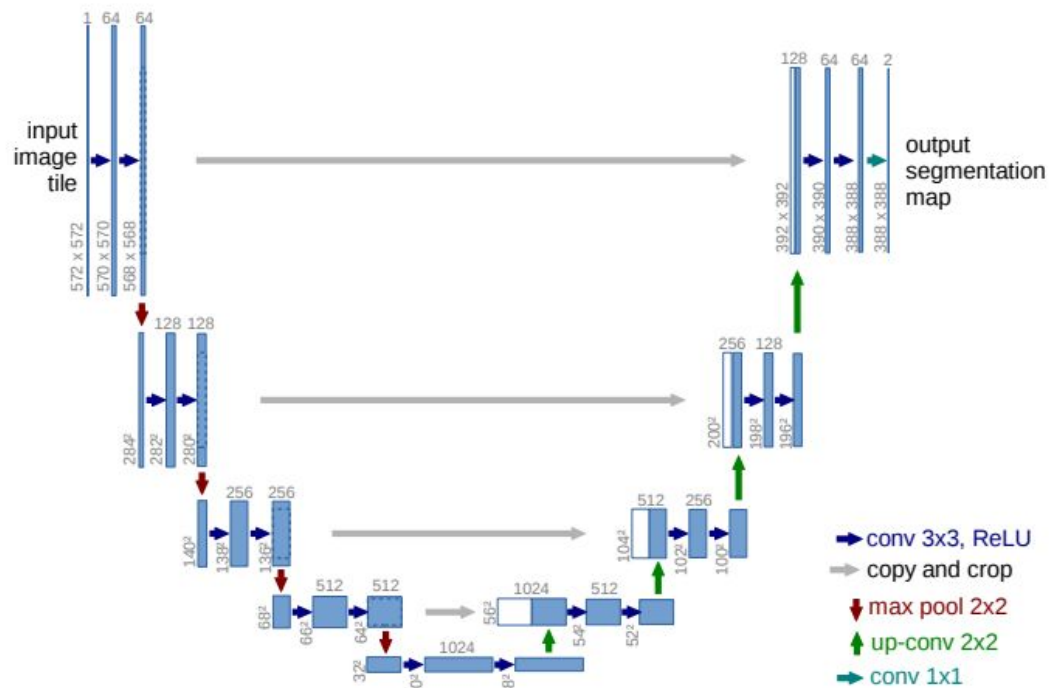
- **Gaussian noise**: se añade a todos los píxeles de la imagen. Tiene distribución de probabilidad normal.
- **Salt and pepper**: un porcentaje de píxeles seleccionados aleatoriamente se cambian a blanco o negro.



Denoising Autoencoders (DAEs) (continuación)

Veamos un ejemplo en `td6-p24-d-dae.ipynb`.

Yapa: Segmentación de imágenes | U-Net



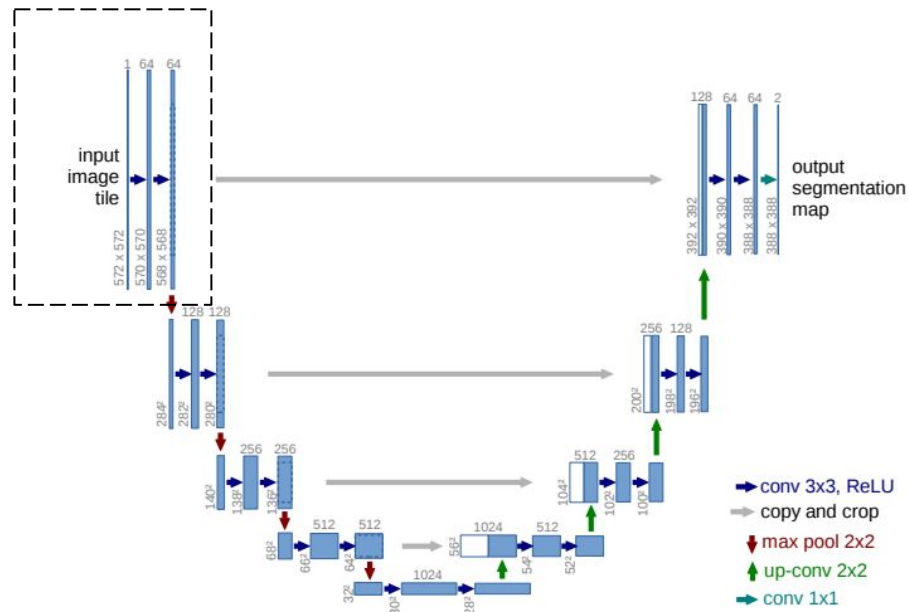
Repasemos la
arquitectura

[Link al paper](#)

Segmentación de imágenes | U-Net

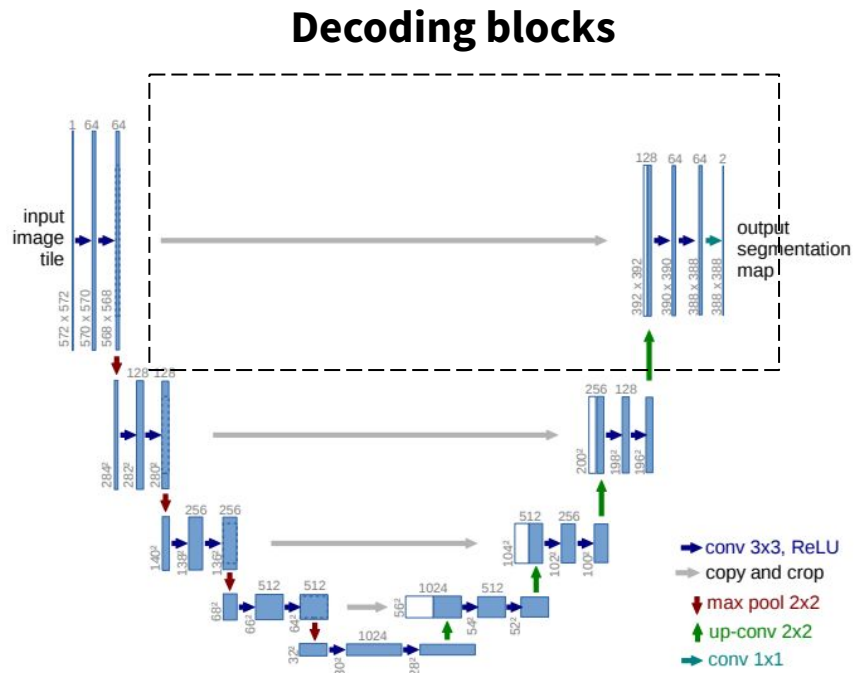
Encoding blocks

Consiste en **dos convoluciones 3x3** (*unpadded convolutions*), seguidas por **ReLU** y **max pooling 2x2** con stride de 2 para *downsampling*. En cada paso de downsampling duplicamos el número de *feature channels*.



Segmentación de imágenes | U-Net

Cada paso en la etapa de expansión consiste en hacer **upsampling** del *feature map* seguido por “**up-convolution**” de 2x2 que duplica el número de canales, luego una **concatenación** con el correspondiente *feature map* recortado de la etapa anterior, y **dos convoluciones 3x3**, cada una seguida por **ReLU**.



Segmentación de imágenes | U-Net

Veamos una implementación en `td6-p24-e-unet.ipynb`.

Cierre

Hoy vimos y practicamos, entre otras cuestiones,

- cómo implementar un **autoencoder** y acceder a sus **espacios latentes**.
- cómo **eliminar ruido** mediante **denoising autoencoders** y
- cómo implementar una arquitectura U-Net para **segmentar imágenes**.

Pueden darnos *feedback* de la clase [acá](#).